Aalto University
School of Electrical
Engineering

Learning from Demonstration

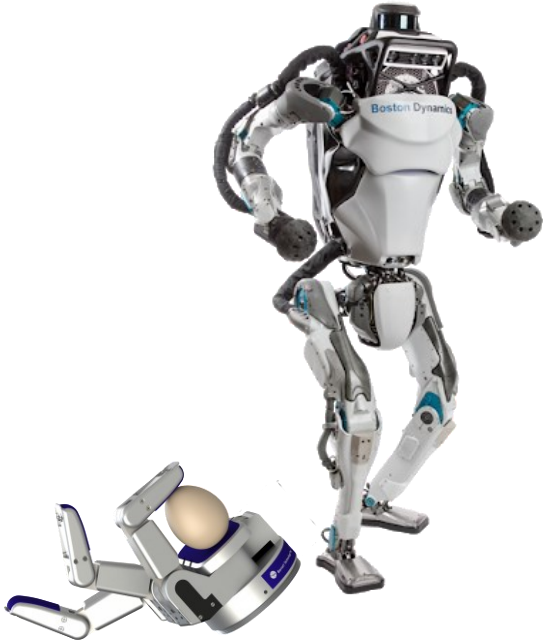
Dynamic Movement Primitives

Fares J. Abu-Dakka, PhD.

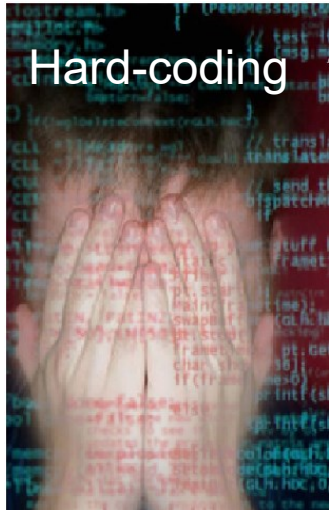
Learning goals

- Understand the idea behind Learning from Demonstration (LfD)
- Understand the formulation of dynamic movement primitives: its
  - *benefits.*
  - *usability.*
  - *etc.*

Introduction:
Background, motivations and challenges



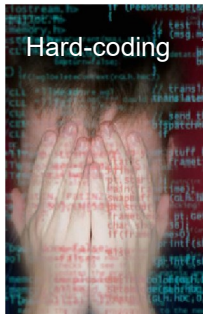
Introduction:
Background, motivations and challenges

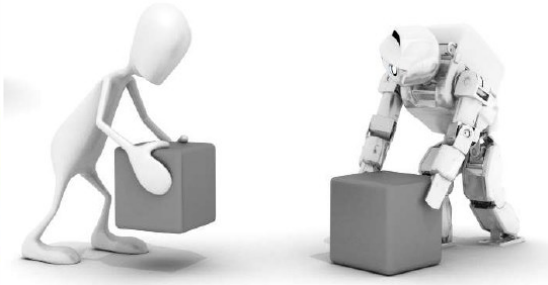


Introduction:
Background, motivations and challenges

Kinesthetic teaching

Imitation Learning

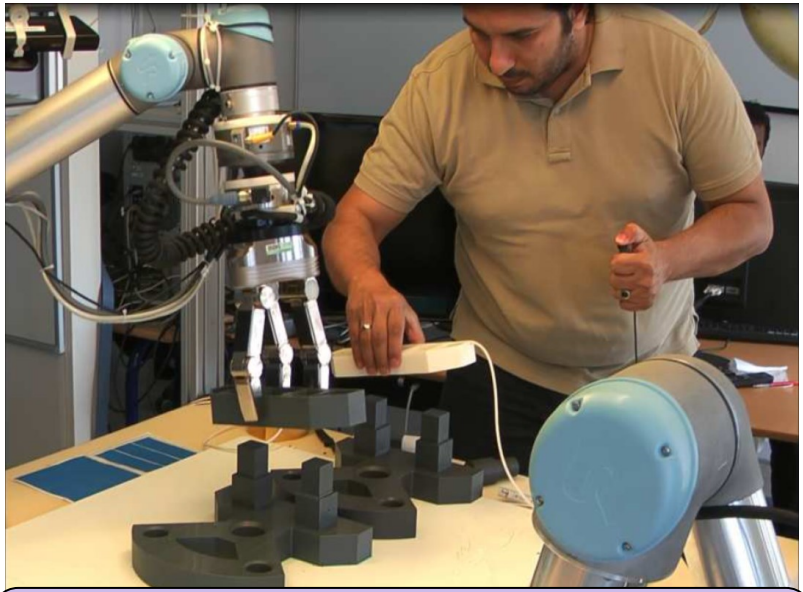




Learning from Demonstration:

Develop new robot behavior through intuitive teaching

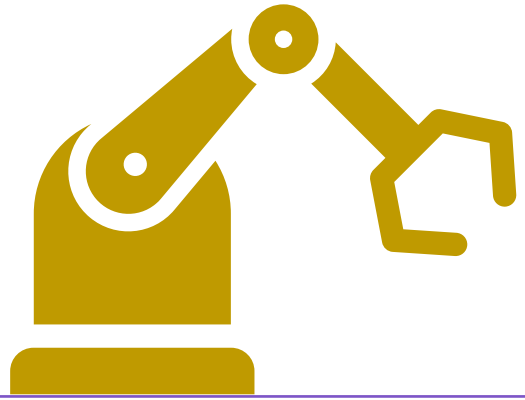
Teleoperation uses a magnetic tracker attached to the object held by human demonstrator.



Kinesthetic guiding uses the robot's gravity compensation mode.

Learning from Demonstration:

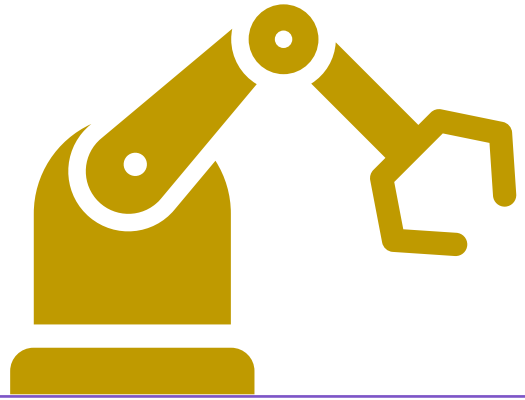
- **Skills** are typically encoded into **parameterized** representations, where parameters are learned from the given demonstrations.
- Skill parameterizations are **compact** (the more the parameters, the more the training data)
- Skill parameterizations are **generalizable** (e.g., different start/goal)



Learning from Demonstration:

Why LfD in industry?

Flexible manufacturing demands for the quick adjustment of the production line in order to automatically produce a variety of different goods.

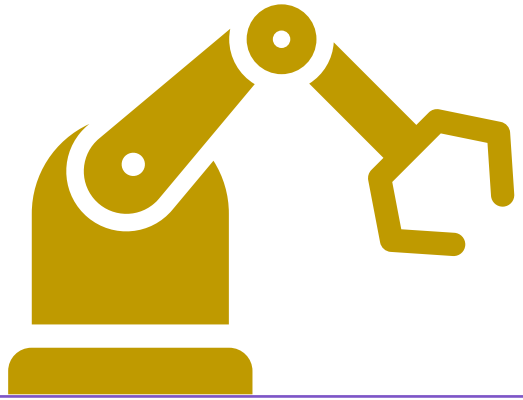


Learning from Demonstration:

Why LfD in industry?

Flexible manufacturing demands for the quick adjustment of the production line in order to automatically produce a variety of different goods.

- Domain expert knowledge
- Hand programming
- Maintain a large/growing code base
- Demonstrating is usually faster than programming
- Generalization is hard to achieve in the standard programming paradigm

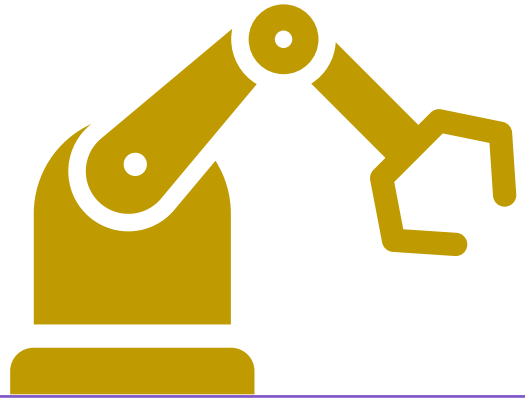


Learning from Demonstration:
Kinesthetic Teaching

Zero-gravity control

- Robot dynamics

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) + f(\dot{q}) = \tau - \tau_{ext}$$



Learning from Demonstration:
Kinesthetic Teaching

Zero-gravity control

- Robot dynamics

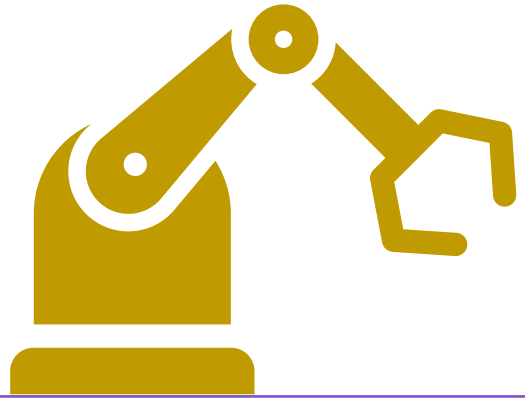
$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) + f(\dot{q}) = \tau - \tau_{ext}$$

- Control input:

$$\tau = C(q,\dot{q})\dot{q} + g(q) + f(\dot{q})$$

- Closed-loop dynamics:

$$\ddot{q} = -M(q)^{-1}\tau_{ext}$$



Learning from Demonstration:
Kinesthetic Teaching

Zero-gravity control

- Robot dynamics

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) + f(\dot{q}) = \tau - \tau_{ext}$$

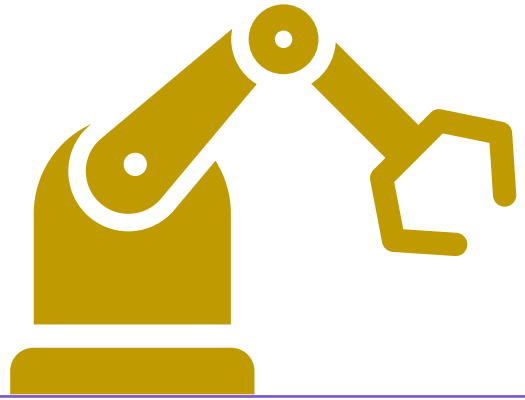
- Control input:

$$\tau = C(q,\dot{q})\dot{q} + g(q) + f(\dot{q})$$

- Closed-loop dynamics:

$$\ddot{q} = -M(q)^{-1}\tau_{ext}$$

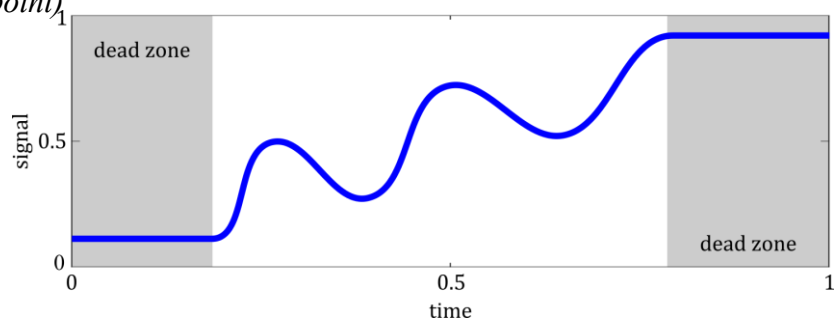
- Safe and reliable human-robot interaction
- Requires access to the torque level (industrial manipulators are usually controlled in position or velocity)
- Available in many collaborative robots (cobots), e.g., using impedance control with low (or zero) gains.

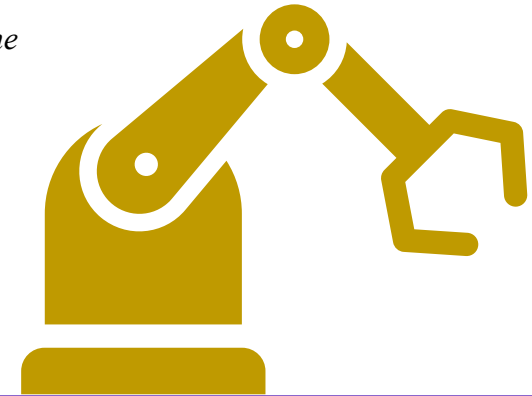


Learning from Demonstration:
Data pre-processing

Dead zones

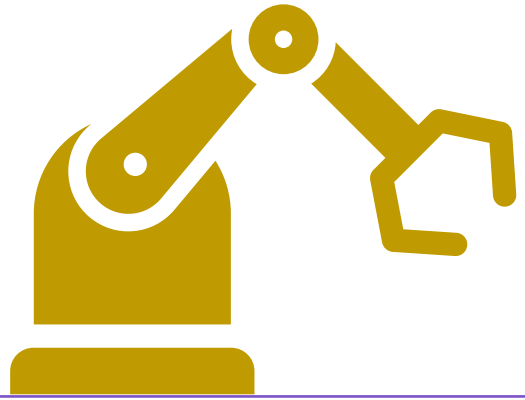
- Parts of the signal that are (almost) constant, i.e., with zero velocity
- Caused by the way of collecting data (put the robot in zero gravity mode and start storing the data)
- Removing them is beneficial for many learning algorithms
  - *Plot the demonstrations against time and manually find cutting points (few demos)*
  - *Remove data points based on a velocity threshold (small velocity → remove the point)*



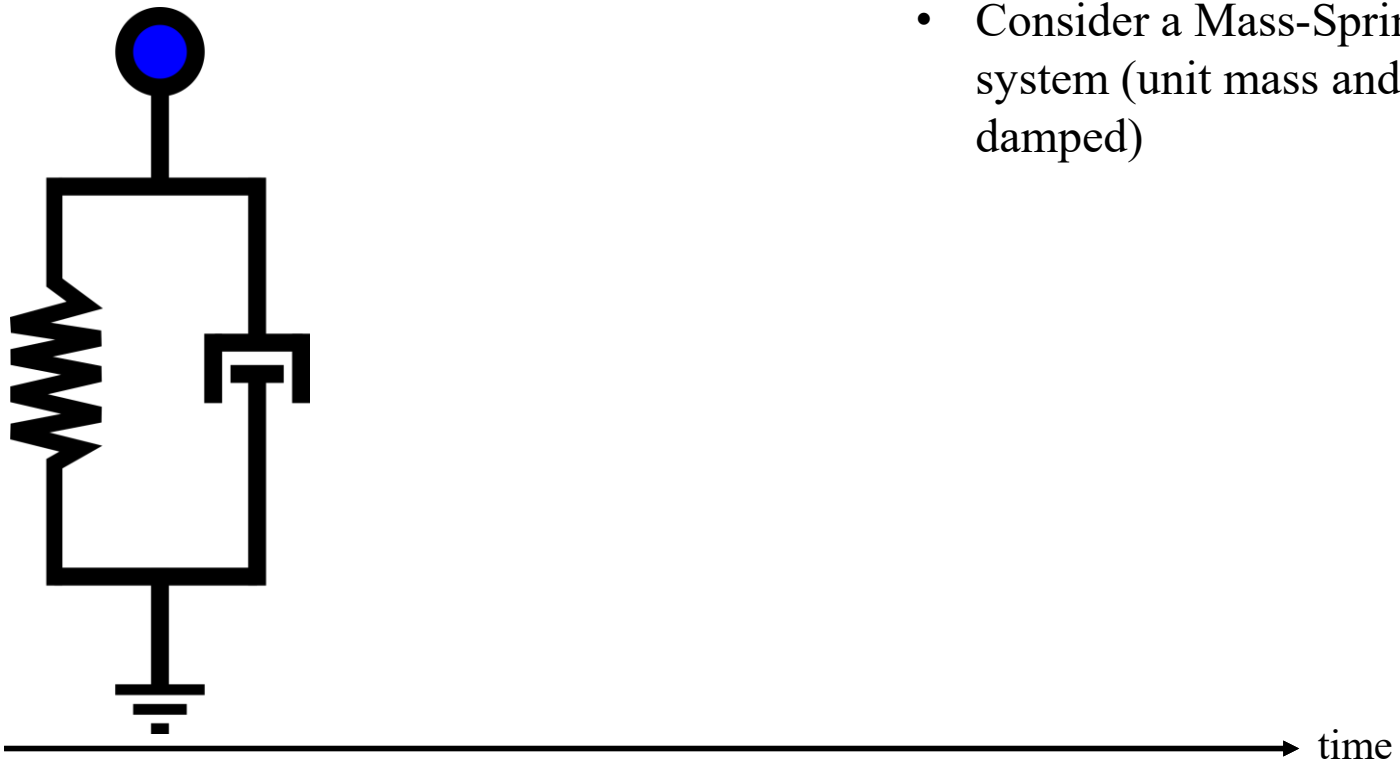


Learning from Demonstration:
Data pre-processing

- Different number of samples
  - Fit a spline and resample
  - Not always needed → Gaussian Mixture Models are robust to different lengths
- Time alignment
  - Use the Dynamic Time Warping if demonstrations are heavily misaligned
  - Not always needed → Hidden Markov Models are robust to time misalignments
- Noise
  - It can be a problem especially when numerical derivatives are needed
  - Apply a low-pass filter, e.g., Butterworth
  - Non always needed → **Dynamic Movement Primitives** are robust to noise

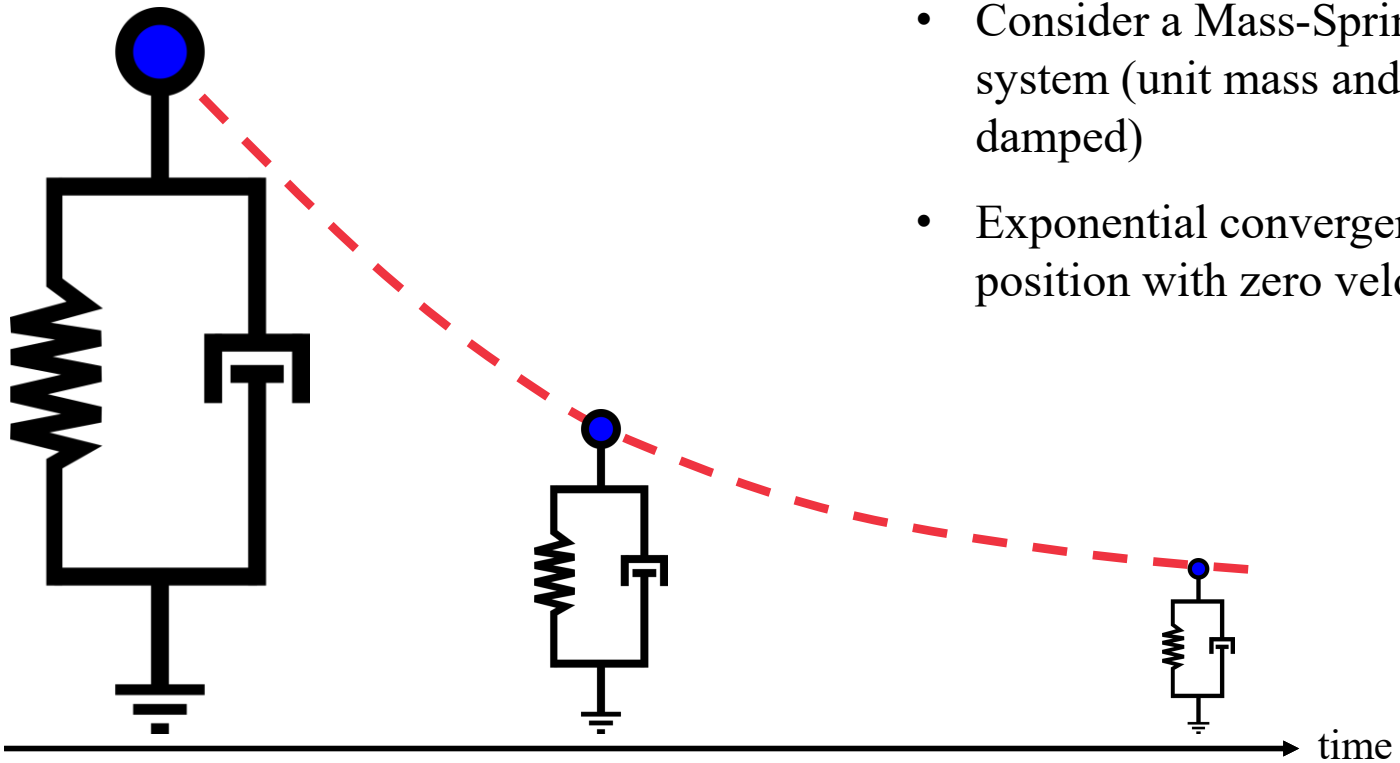


Dynamic Movement Primitives (DMPs) Idea



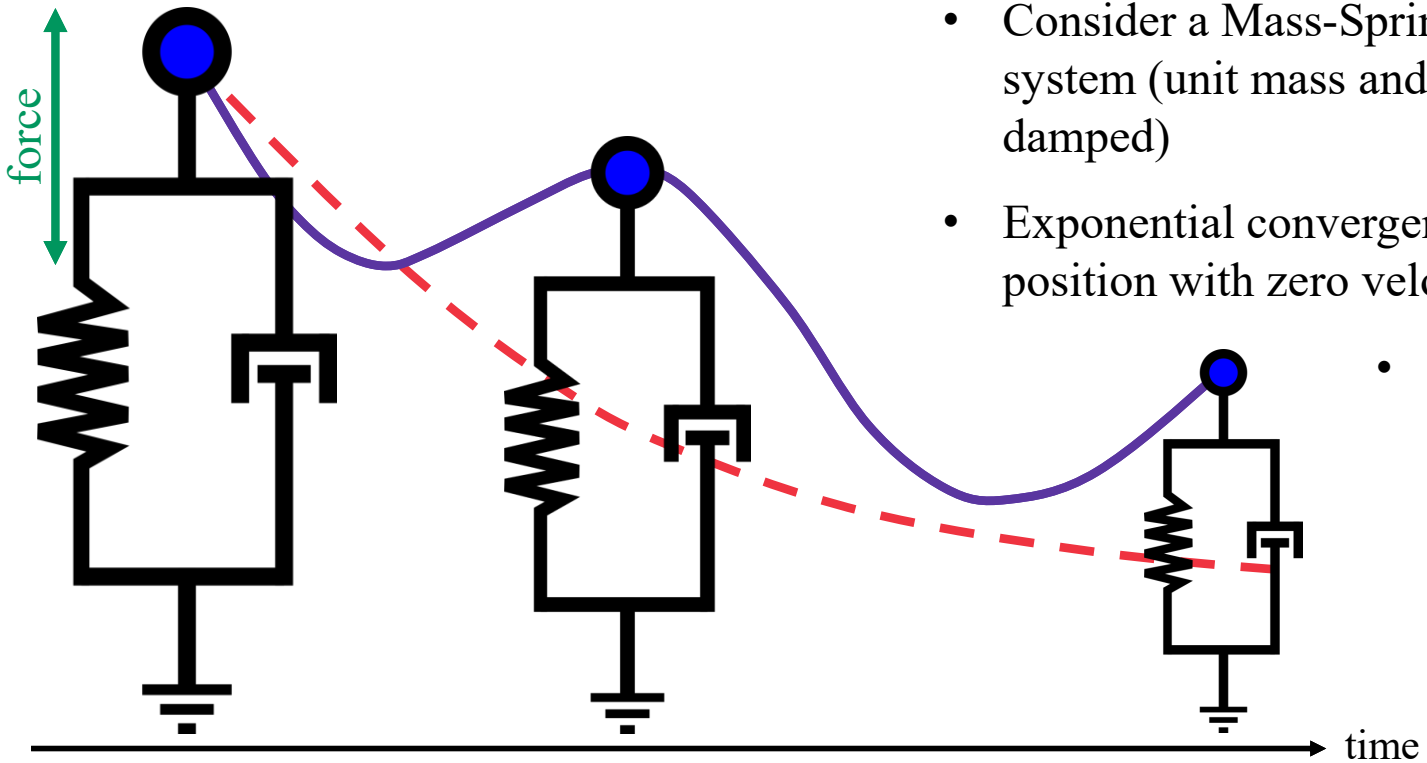
- Consider a Mass-Spring-Damper system (unit mass and critically damped)

Dynamic Movement Primitives (DMPs) Idea



- Consider a Mass-Spring-Damper system (unit mass and critically damped)
- Exponential convergence to the rest position with zero velocity

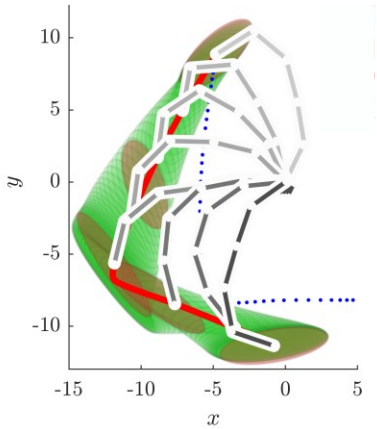
Dynamic Movement Primitives (DMPs) Idea



- Consider a Mass-Spring-Damper system (unit mass and critically damped)
- Exponential convergence to the rest position with zero velocity
- Add a **forcing term** to follow different trajectories

Discrete Motion
Spring-damper dynamics

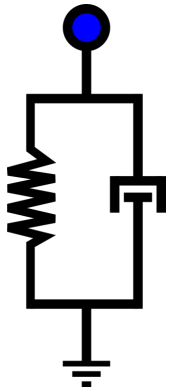
- **Discrete motion:** motion with a fixed start and goal **(point-to-point)**



Discrete Motion
Spring-damper dynamics

- **Discrete motion:** motion with a fixed start and goal **(point-to-point)**
- Consider a simple spring-damper dynamics (unit mass)

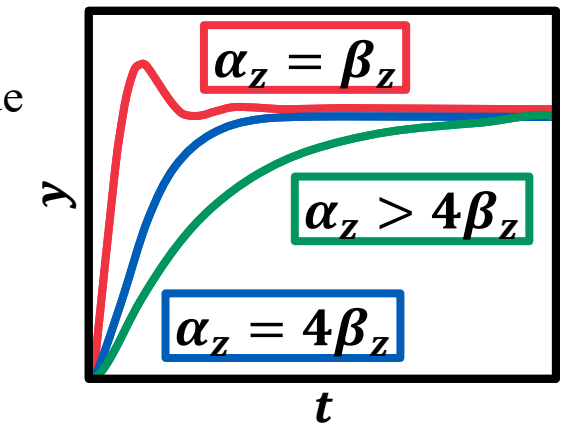
$$\tau\,\ddot{y} = \alpha_z(\beta_z(g - y) - \dot{y})$$



- y : Position (single joint or end-effector direction)
- $\dot{y}$:Velocity (single joint or end-effector direction)
- $\ddot{y}$: Acceleration (single joint or end-effector direction)
- g : Goal (desired attractor position)
- $\tau > 0$: Time scaling (generate slower ($\tau > 1$) or faster ($\tau < 1$) motion)
- α_z, β_z : Control gains (globally asymptotically stable if $\alpha_z, \beta_z > 0$)

Spring-Damper Dynamics
Why a second-order dynamics?

- Many robot controllers require desired position, velocity, and acceleration
- Generate desired trajectories with smooth velocities
- α_z and $\beta_z > 0$ define the behavior of the 2nd order system. To avoid oscillation $\rightarrow \alpha_z = 4\,\beta_z$ for critically damped system
- Critically damped system with τ, α_z, $\beta_z > 0$, the convergence of the underlying dynamic system to a unique attractor point at $y = g$, $z = 0$ is ensured.



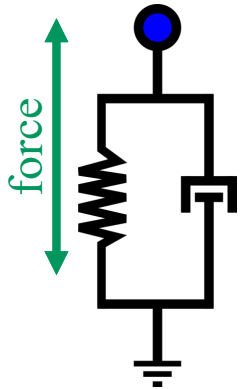
- **Limitations**
- Motion is fixed from start to goal (point-to-point).
- Motion is exponential.

Spring-Damper Dynamics
Limitations

- **Discrete motion**: Motion is fixed from start to goal (**point-to-point**).
- **Spring-damper** system: Motion is exponential.

$$\tau\,\ddot{y} = \alpha_z(\beta_z(g - y) - \dot{y}) + f$$

- f : Nonlinear forcing term
  - Helps to generate diverse and more complex trajectory (**not only** exponential)
  - It can be learned from a single trajectory demonstration.
- Problem → The superposition of two vector fields **does not** preserve, in general, the **stability**.



Spring-Damper Dynamics
Stability

$$\tau\,\ddot{y} = \alpha_z(\beta_z(g - y) - \dot{y}) + f$$

- f : Nonlinear forcing term
  - Helps to generate diverse and more complex trajectory (**not only** exponential)
  - It can be learned from a single trajectory demonstration.
- Problem → The superposition of two vector fields **does not** preserve, in general, the **stability**.
- Solution → Make the forcing term vanishing with time.

$$\tau\,\ddot{y} = \alpha_z(\beta_z(g - y) - \dot{y}) + f(t)$$
$$\lim_{t \to \infty} f(t) = 0$$

→

Stable asymptotic dynamic (α_z , β_z > 0)

$$\tau\,\ddot{y} = \alpha_z(\beta_z(g - y) - \dot{y}) + f(t)$$

Spring-Damper Dynamics
Time dependency

- The time dependent forcing term renders the forces spring-damper a **linear time-variant** dynamics
  - PROS:
    - It can genrate complex trajectories
    - It is globally asymptotically stable
    - The goal can be easily changed (replace g with g')
  - CONS:
    - Synchronization of multiple systems is not easy to achieve (useful to control multiple degrees-of-freedom (DoF)
    - It is not easy to include additional coupling terms
    - Time evolution cannot be controlled via adjustable parameters

Discrete DMP
Formulation

- **Transformation System**

$$\tau\,\dot{z} = \alpha_z(\beta_z(g - y) - z) + f(x)$$

$$\tau\dot{y} = z$$

  - The transformation system is a set of first-order differential equations obtained from the forced spring-damper system by the change of variables $\dot{y} = z$

Discrete DMP
Formulation

- **Transformation System**

$$\tau\,\dot{z} = \alpha_z(\beta_z(g - y) - z) + f(x)$$

$$\tau\dot{y} = z$$

  - The transformation system is a set of first-order differential equations obtained from the forced spring-damper system by the change of variables $\dot{y} = z$

- **Canonical system:** $\tau\dot{x} = -\alpha_x x$

  - The canonical system vanishes (exponentially) for $t \to \infty$ if $\alpha_x > 0$

Discrete DMP
Formulation

- **Transformation System**

$$\tau\,\dot{z} = \alpha_z(\beta_z(g - y) - z) + f(x)$$

$$\tau\dot{y} = z$$

  - The transformation system is a set of first-order differential equations obtained from the forced spring-damper system by the change of variables $\dot{y} = z$

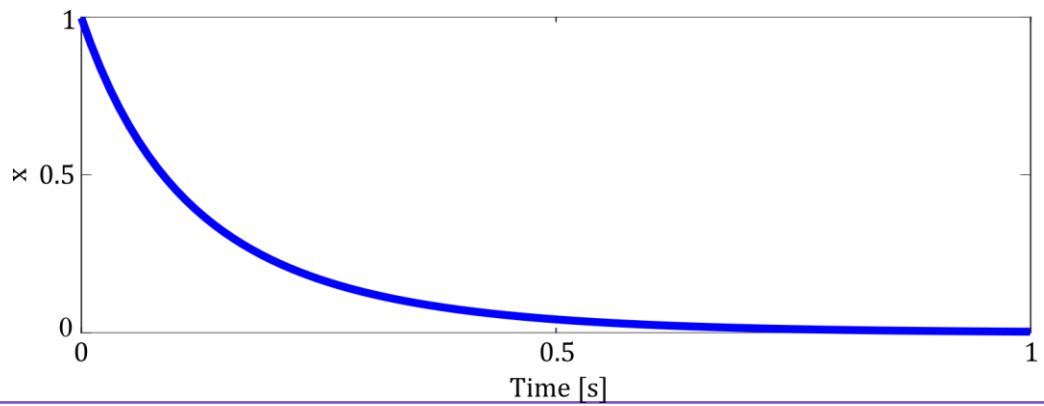
- **Canonical system:** $\tau\dot{x} = -\alpha_x x$
  - The canonical system vanishes (exponentially) for $t \to \infty$ if $\alpha_x > 0$
  - The **nonlinear forcing term** was classically parametrized as

$$f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0), \qquad \psi_i(x) = \exp(-h_i(x - c_i)^2),$$

Discrete DMP
Canonical system

$$\tau \dot{x} = -\alpha_x x$$

- The canonical system vanishes (exponentially) for $t \to \infty$ if $\alpha_x > 0$
- Initialized as $x(0) = x_0 = 1$ (other choices as possible)
  - $x \approx 1$ → The motion (time evolution) has just started.
  - $x \approx 0$ → The goal has been almost reached



Discrete DMP
Forcing term

- The **nonlinear forcing term** was classically parametrized as a Radial Basis Function Network

$$f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0), \qquad \psi_i(x) = \exp(-h_i(x - c_i)^2),$$

- Weighted summation of Gaussian basis functions
- For $t \to \infty$, $x \to 0$, $f(x) \to 0$
- Scaling by $g - y_0$ (for now $g \neq y_0$)
  - Prevents large acceleration at the beginning of the motion
  - Preserves scale invariance

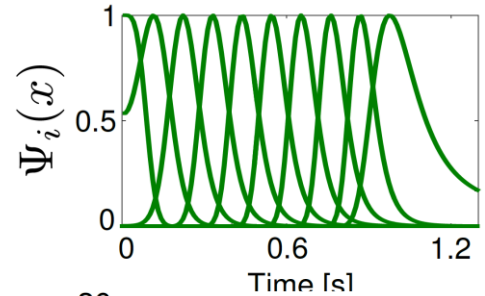
Discrete DMP

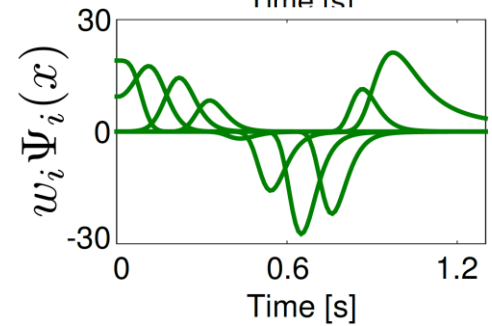
Forcing term

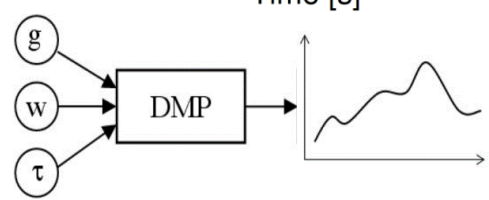
$$f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0),$$

$$\psi_i(x) = \exp(-h_i(x - c_i)^2),$$

RBF →





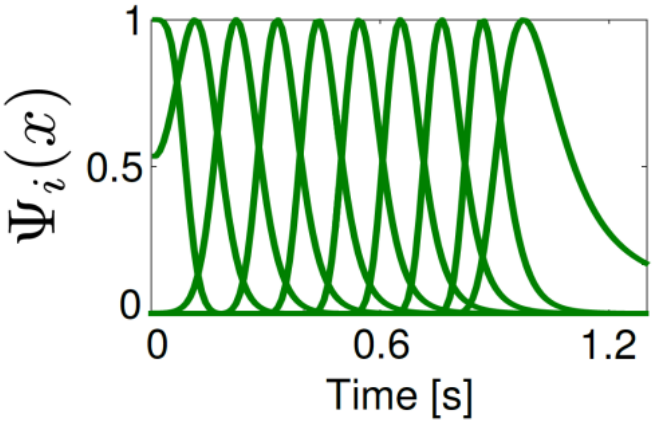


- Fixing h_i, c_i (widths and centers of the RBF) → the learning problem is linear in the weights w_i,
- encodes the desired additional acceleration profile,
- learnable function,
- enables the robot to follow any smooth trajectory from the initial position y_0 to the final configuration g.

Discrete DMP
Basis functions

$$f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0), \qquad \psi_i(x) = \exp(-h_i(x - c_i)^2),$$

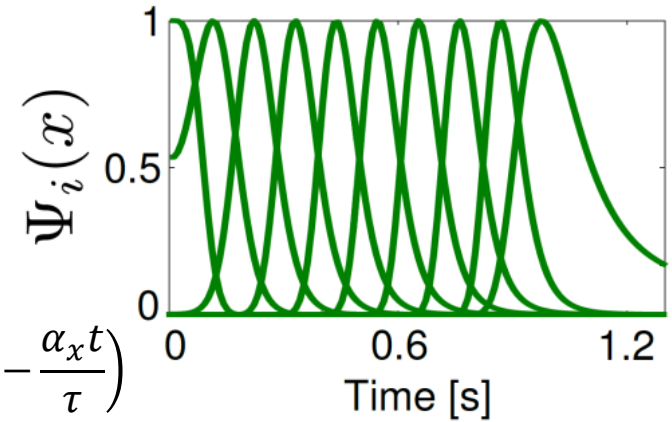
- Problem: How do we fix widths and centers?
- Solution: Make them equally spaced in time.



Discrete DMP
Basis functions

$$f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0), \qquad \psi_i(x) = \exp(-h_i(x - c_i)^2),$$

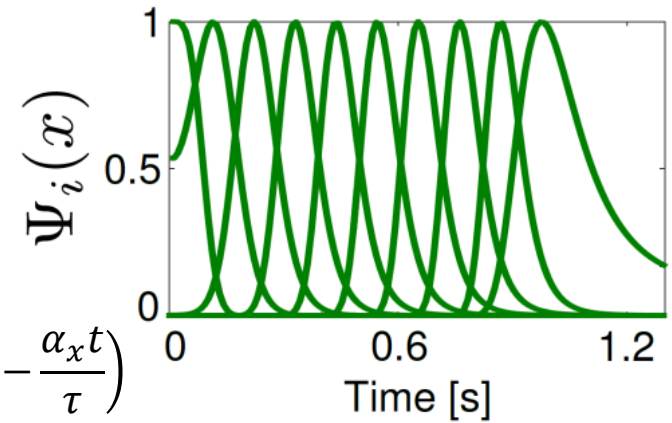
- Problem: How do we fix widths and centers?
- Solution: Make them equally spaced in time.
- Exponentially spaced in x



$$\tau \dot{x} = -\alpha_x x \;\rightarrow x(t) = \exp\left(-\frac{\alpha_x t}{\tau}\right)$$

Discrete DMP
Basis functions

$$f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0), \qquad \psi_i(x) = \exp(-h_i(x - c_i)^2),$$



- Problem: How do we fix widths and centers?
- Solution: Make them equally spaced in time.
- Exponentially spaced in x

$$\tau \dot{x} = -\alpha_x x \rightarrow x(t) = \exp\left(-\frac{\alpha_x t}{\tau}\right)$$

- Set $\tau = T$ (total duration of the motion) → $0 \leq \frac{t}{\tau} \leq 1$
- For given N: $c_i = \exp\left(-\alpha_x \frac{i-1}{N-1}\right)$ and $h_i = \frac{1}{(c_{i+1} - c_i)^2}$, $h_{N-1} = h_N$, $i = 1, \ldots, N$

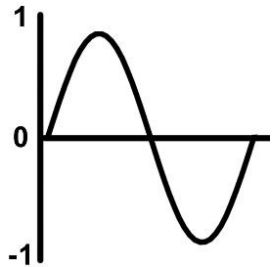
Periodic DMP
Formulation

- Converges towards a limit cycle attractor
- **Transformation System**

$$\tau\,\dot{z} = \alpha_z(\beta_z(g - y) - z) + f(\phi, r)$$

$$\tau\dot{y} = z$$

  - g is the mean (set point) of the oscillation, e.g., $g = 0$ for sinusoid.
  - The oscillation frequency is changed by τ
    (e.g., bigger τ → smaller frequency → bigger period)

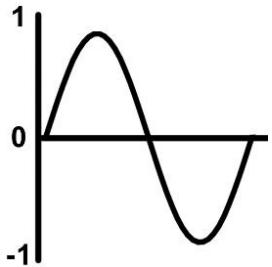


Periodic DMP
Formulation

- Converges towards a limit cycle attractor
- **Transformation System**

$$\tau\,\dot{z} = \alpha_z(\beta_z(g - y) - z) + f(\phi, r)$$

$$\tau\dot{y} = z$$



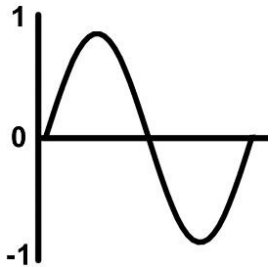
  - g is the mean (set point) of the oscillation, e.g., $g = 0$ for sinusoid.
  - The oscillation frequency is changed by τ
    (e.g., bigger τ → smaller frequency → bigger period)
- **Canonical system:** $\tau\dot{\phi} = 1$
  - The phase ϕ grows linearly with time

Periodic DMP
Formulation

- Converges towards a limit cycle attractor
- **Transformation System**

$$\tau\,\dot{z} = \alpha_z(\beta_z(g - y) - z) + f(\phi, r)$$

$$\tau\dot{y} = z$$



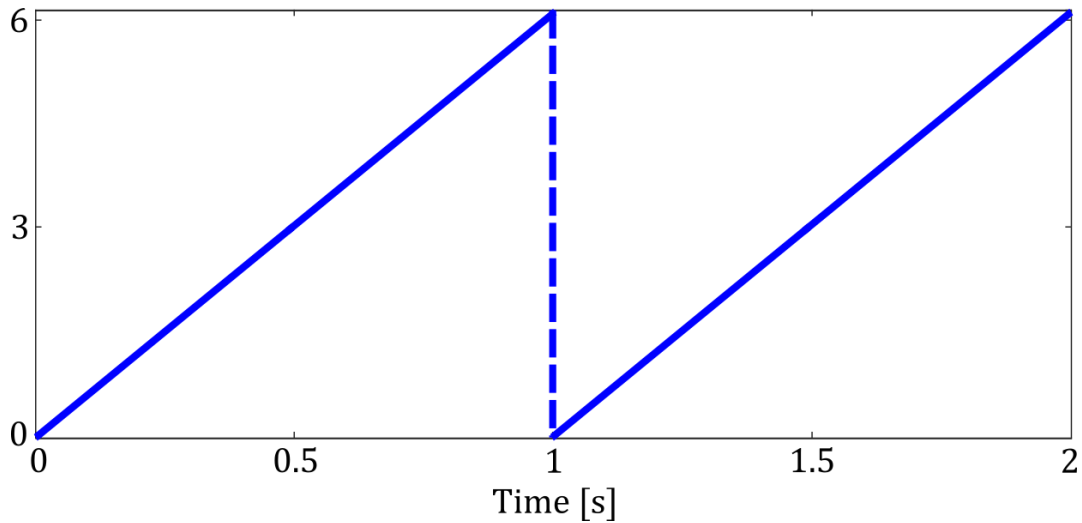
  - g is the mean (set point) of the oscillation, e.g., $g = 0$ for sinusoid.
  - The oscillation frequency is changed by τ
    (e.g., bigger τ → smaller frequency → bigger period)
- **Canonical system:** $\tau\dot{\phi} = 1$
  - The phase ϕ grows linearly with time
- The nonlinear forcing term is parametrized as

$$f(\phi, r) = \frac{\sum_{i=1}^{N} w_i \psi_i(\phi)}{\sum_{i=1}^{N} \psi_i(\phi)} r, \qquad \psi_i(\phi) = \exp(-h_i(\cos(\phi - c_i) - 1)),$$

Periodic DMP
Canonical system

$$\tau\dot{\phi} = 1$$

- The phase ϕ grows linearly with time
- Periodicity is given by the choice of the basis functions



Periodic DMP
Forcing term

$$f(\phi, r) = \frac{\sum_{i=1}^{N} w_i \psi_i(\phi)}{\sum_{i=1}^{N} \psi_i(\phi)} r, \qquad \psi_i(\phi) = \exp(-h_i(\cos(\phi - c_i) - 1)),$$

- Weighted summation of Von Mises basis functions
  (a periodic extension of Gaussian distribution, i.e., Gaussian on circle)
- The amplitude r is used to modulate the amplitude of the periodic signal. Set $r = 1$ if unused
- h_i, c_i are fixed (linear in w_i), e.g., linearly spaced in the phase interval $[0,2\pi]$ → Obtained with the same approach used for discrete DMPs

DMP
Motion generation

Given:
$\alpha_z, \beta_z, g, x_0, y_0, z_0, h_i, c_i, w_i, \Delta t, \epsilon$

1. Initialize: $x_t = x_0, y_t = y_0, z_t = z_0$
2. Compute $f(x_t)$
3. Compute $\dot{z}_t = \frac{1}{\tau}(\alpha_z(\beta_z(g - y_t) - z_t) + f(x_t))$
4. Euler integration

$$z_t = z_t + \dot{z}_t \Delta t$$
$$y_t = y_t + \frac{z_t}{\tau}\Delta t$$
$$x_t = x_t - \frac{\alpha_x}{\tau} x_t \Delta t$$

5. Repeat 2 to 4 until convergence

$$\|g - y_t\|^2 < \epsilon$$

DMP
Motion generation

Given:
$\alpha_z, \beta_z, g, x_0, y_0, z_0, h_i, c_i, w_i, \Delta t, \epsilon$

1. Initialize: $x_t = x_0, y_t = y_0, z_t = z_0$
2. Compute $f(x_t)$
3. Compute $\dot{z}_t = \frac{1}{\tau}(\alpha_z(\beta_z(g - y_t) - z_t) + f(x_t))$
4. Euler integration
$$z_t = z_t + \dot{z}_t \Delta t$$
$$y_t = y_t + \frac{z_t}{\tau}\Delta t$$
$$x_t = x_t - \frac{\alpha_x}{\tau} x_t \Delta t$$
5. Repeat 2 to 4 until convergence
$$\|g - y_t\|^2 < \epsilon$$

Given:
$\alpha_z, \beta_z, g, r, \phi_0, y_0, z_0, h_i, c_i, w_i, \Delta t$

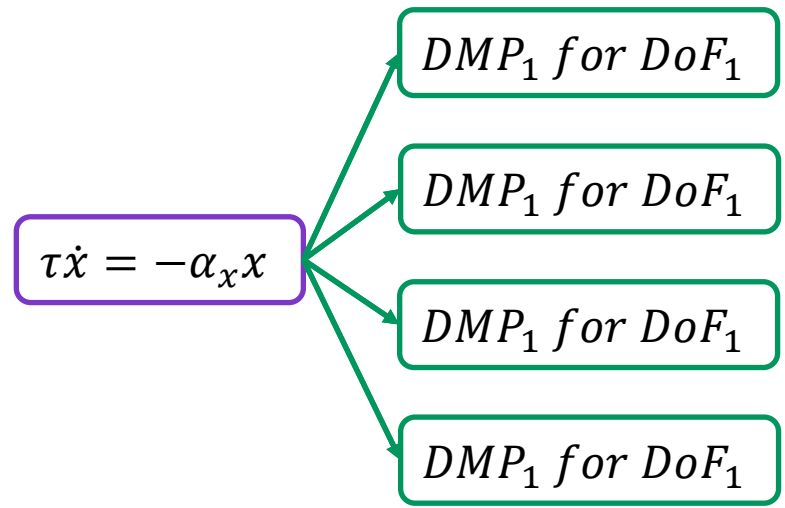
1. Initialize: $\phi_t = \phi_0, y_t = y_0, z_t = z_0$
2. Compute $f(\phi_t, r)$
3. Compute $\dot{z}_t = \frac{1}{\tau}(\alpha_z(\beta_z(g - y_t) - z_t) + f(\phi_t, r))$
4. Euler integration
$$z_t = z_t + \dot{z}_t \Delta t$$
$$y_t = y_t + \frac{z_t}{\tau}\Delta t$$
$$\phi_t = \phi_t + \frac{\Delta t}{\tau}$$
5. Repeat 2 to 4 (e.g., for P periods)

DMPs
Multiple Degrees-of-Freedom (DoF)

- We have introduced the DMP formulation for a single DoF. However, in order to plan trajectories for the robot (in joint or Cartesian space), we need to control multiple DoF.

DMPs
Multiple Degrees-of-Freedom (DoF)

- We have introduced the DMP formulation for a single DoF. However, in order to plan trajectories for the robot (in joint or Cartesian space), we need to control multiple DoF.
- We can achieve this with DMPs:
  - Consider a separate transformation system for controlling each DoF
  - Consider a single canonical system, shared across all the transformation systems.
    It synchronizes the time evolution between all transformation systems.



DMPs
Learning from Demonstration

- Input: desired positions $y_{d,t}$, velocities $\dot{y}_{d,t}$, and accelerations $\ddot{y}_{d,t}$, $t = 1, \dots, T$, e.g., from kinesthetic demonstration
  (velocity and acceleration can be estimated from $y_{d,t}$ via numerical differentiation given Δt)

DMPs
Learning from Demonstration

- Input: desired positions $y_{d,t}$, velocities $\dot{y}_{d,t}$, and accelerations $\ddot{y}_{d,t}$, $t = 1, \dots, T$, e.g., from kinesthetic demonstration
  (velocity and acceleration can be estimated from $y_{d,t}$ via numerical differentiation given Δt)
- Initialize all parameters
$$f(x) = \tau\,\dot{z} - \alpha_z(\beta_z(g - y) - z)$$
- Insert the demonstration and stack the forcing terms in a vector
$$\mathbf{f} = \begin{bmatrix} f_1 = \tau^2\,\ddot{y}_{d,1} - \alpha_z\big(\beta_z\big(g - y_{d,1}\big) - \dot{y}_{d,1}\big) \\ \vdots \\ f_T = \tau^2\,\ddot{y}_{d,T} - \alpha_z\big(\beta_z\big(g - y_{d,T}\big) - \dot{y}_{d,T}\big) \end{bmatrix}$$

DMPs
Learning from Demonstration

- Compute

$$\mathbf{\Psi} = (g - y_{d,0})\begin{bmatrix} \frac{\psi_1(x_1)}{\sum_{i=1}^{N}\psi_i(x_1)}x_1 & \cdots & \frac{\psi_1(x_1)}{\sum_{i=1}^{N}\psi_i(x_1)}x_1 \\ \vdots & \ddots & \vdots \\ \frac{\psi_T(x_T)}{\sum_{i=1}^{N}\psi_i(x_T)}x_T & \cdots & \frac{\psi_T(x_T)}{\sum_{i=1}^{N}\psi_i(x_T)}x_T \end{bmatrix}, \mathbf{\Psi} = r\begin{bmatrix} \frac{\psi_1(\phi_1)}{\sum_{i=1}^{N}\psi_i(\phi_1)} & \cdots & \frac{\psi_1(\phi_1)}{\sum_{i=1}^{N}\psi_i(x_1)}x_1 \\ \vdots & \ddots & \vdots \\ \frac{\psi_T(\phi_T)}{\sum_{i=1}^{N}\psi_i(\phi_T)} & \cdots & \frac{\psi_T(\phi_T)}{\sum_{i=1}^{N}\psi_i(\phi_T)} \end{bmatrix}$$

- Stack the weights in a vector $\mathbf{w} = [w_1, w_2, \dots, w_N]^{\mathrm{T}}$

DMPs
Learning from Demonstration

- Compute

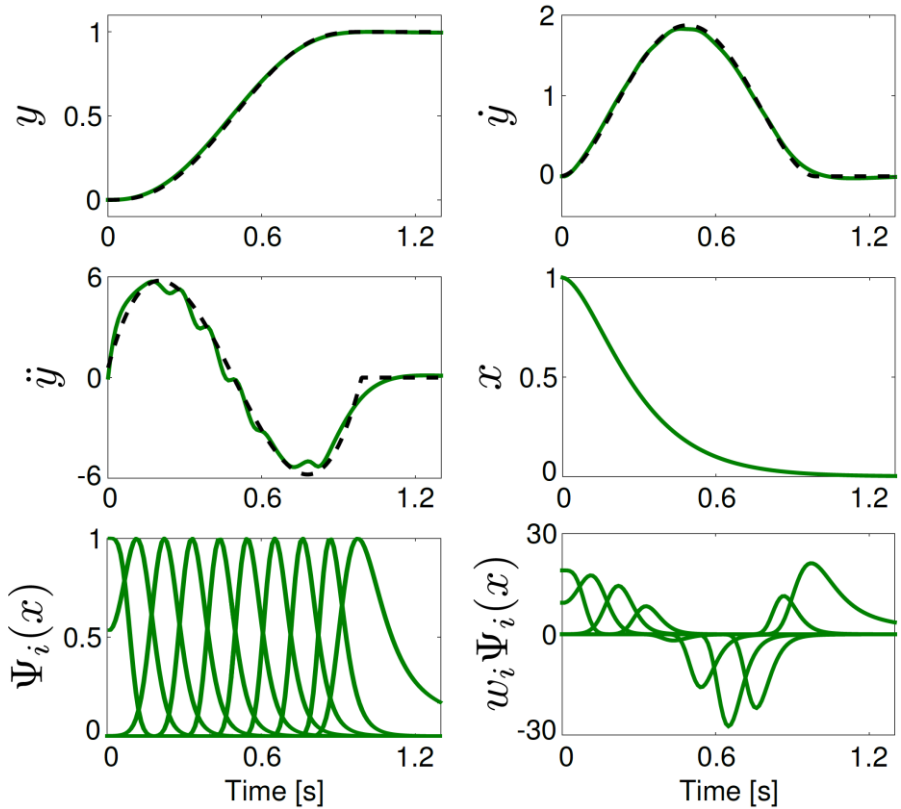
$$\mathbf{\Psi} = (g - y_{d,0})\begin{bmatrix} \frac{\psi_1(x_1)}{\sum_{i=1}^{N}\psi_i(x_1)}x_1 & \cdots & \frac{\psi_1(x_1)}{\sum_{i=1}^{N}\psi_i(x_1)}x_1 \\ \vdots & \ddots & \vdots \\ \frac{\psi_T(x_T)}{\sum_{i=1}^{N}\psi_i(x_T)}x_T & \cdots & \frac{\psi_T(x_T)}{\sum_{i=1}^{N}\psi_i(x_T)}x_T \end{bmatrix}, \mathbf{\Psi} = r\begin{bmatrix} \frac{\psi_1(\phi_1)}{\sum_{i=1}^{N}\psi_i(\phi_1)} & \cdots & \frac{\psi_1(\phi_1)}{\sum_{i=1}^{N}\psi_i(x_1)}x_1 \\ \vdots & \ddots & \vdots \\ \frac{\psi_T(\phi_T)}{\sum_{i=1}^{N}\psi_i(\phi_T)} & \cdots & \frac{\psi_T(\phi_T)}{\sum_{i=1}^{N}\psi_i(\phi_T)} \end{bmatrix}$$

- Stack the weights in a vector $\mathbf{w} = [w_1, w_2, \dots, w_N]^{\mathrm{T}}$
- Solve: $\mathbf{\Psi w} = \mathbf{f}$
  by minimizing the difference between $\mathbf{\Psi w}$ and $\mathbf{f}$, i.e., $J = \|\mathbf{f} - \mathbf{\Psi w}\|$.
  Least squares solution:

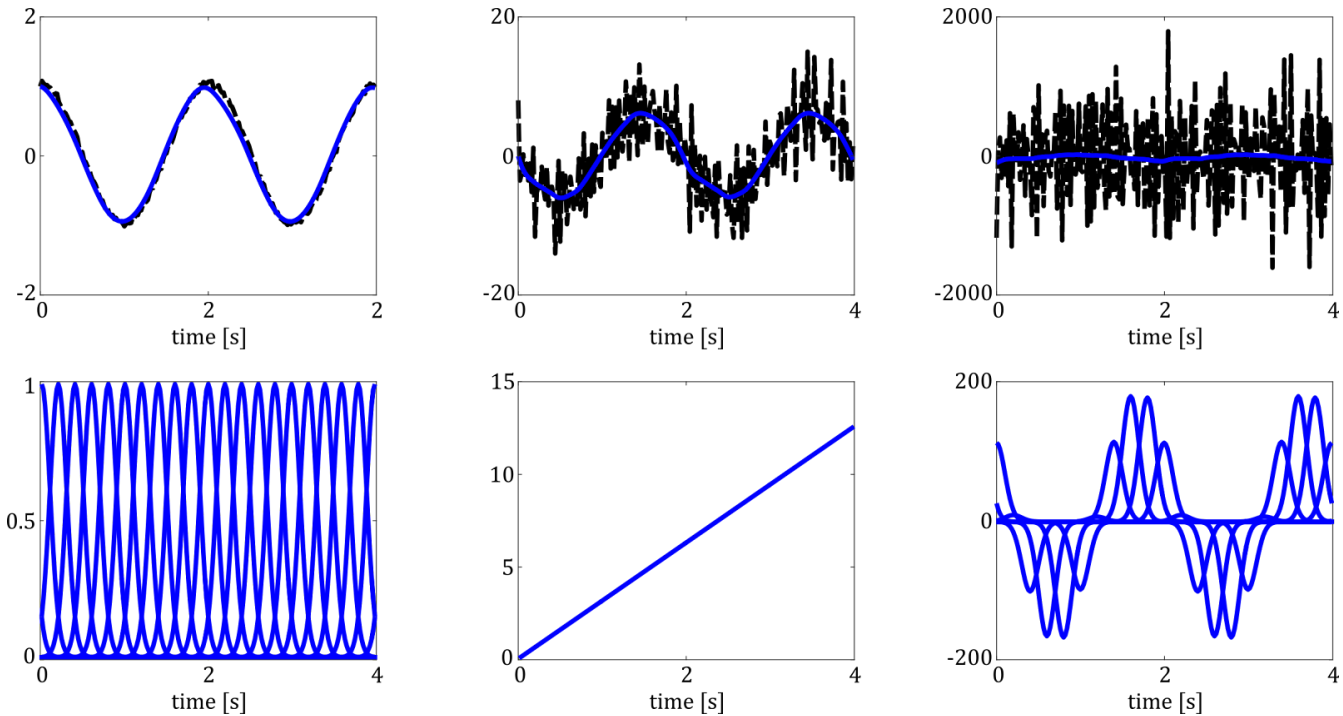
$$\mathbf{w} = \mathbf{\Psi}^{+}\mathbf{f}$$

  where $\mathbf{\Psi}^{+} = \left(\mathbf{\Psi}^{\mathrm{T}}\mathbf{\Psi} + \lambda\mathbf{I}\right)^{-1}\mathbf{\Psi}^{\mathrm{T}}$

Discrete DMP
Example



Periodic DMP
Example



Discrete DMP
Issues

- In the classical formulation $f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0)$ where $(g - y_0)$ is used to ensure scale invariance and avoid high acceleration
- What happens **if** $\boldsymbol{g = y_0}$, e.g., height coordinate in pick-and-place?

Discrete DMP
Issues

- In the classical formulation $f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0)$ where $(g - y_0)$ is used to ensure scale invariance and avoid high acceleration
- What happens **if** $\boldsymbol{g} = \boldsymbol{y_0}$, e.g., height coordinate in pick-and-place?
  - $f(x) = 0, \mathrm{z}_0 = 0 \rightarrow \tau\, \dot{z}_0 = \alpha_z(\beta_z(g - y) - 0) + 0 = 0 \rightarrow$ motion does not start!

Discrete DMP
Issues

- In the classical formulation $f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0)$ where $(g - y_0)$ is used to ensure scale invariance and avoid high acceleration
- What happens **if $\boldsymbol{g} = \boldsymbol{y_0}$**, e.g., height coordinate in pick-and-place?
  - $f(x) = 0, \mathrm{z}_0 = 0 \rightarrow \tau\, \dot{z}_0 = \alpha_z(\beta_z(g - y) - 0) + 0 = 0 \rightarrow$ motion does not start!
- What happens if $g \approx \mathrm{y}_0$?
  - The term $(g - \mathrm{y}_0) \approx 0 \rightarrow w_i \rightarrow \infty$ to reproduce the desired force $\rightarrow$ Large overshoot when you change the goal!

Discrete DMP
Issues

- In the classical formulation $f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0)$ where $(g - y_0)$ is used to ensure scale invariance and avoid high acceleration
- What happens **if $\boldsymbol{g = y_0}$**, e.g., height coordinate in pick-and-place?
  - $f(x) = 0, z_0 = 0 \rightarrow \tau\, \dot{z}_0 = \alpha_z(\beta_z(g - y) - 0) + 0 = 0 \rightarrow$ motion does not start!
- What happens if $g \approx y_0$?
  - The term $(g - y_0) \approx 0 \rightarrow w_i \rightarrow \infty$ to reproduce the desired force $\rightarrow$ Large overshoot when you change the goal!
- Solution: $f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x \cancel{(g - y_0)} \rightarrow f'(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x$

$$\tau\, \dot{z}_0 = \alpha_z(\beta_z(g - y) - z) + (g - y_0)x + f'(x)$$

Discrete DMP

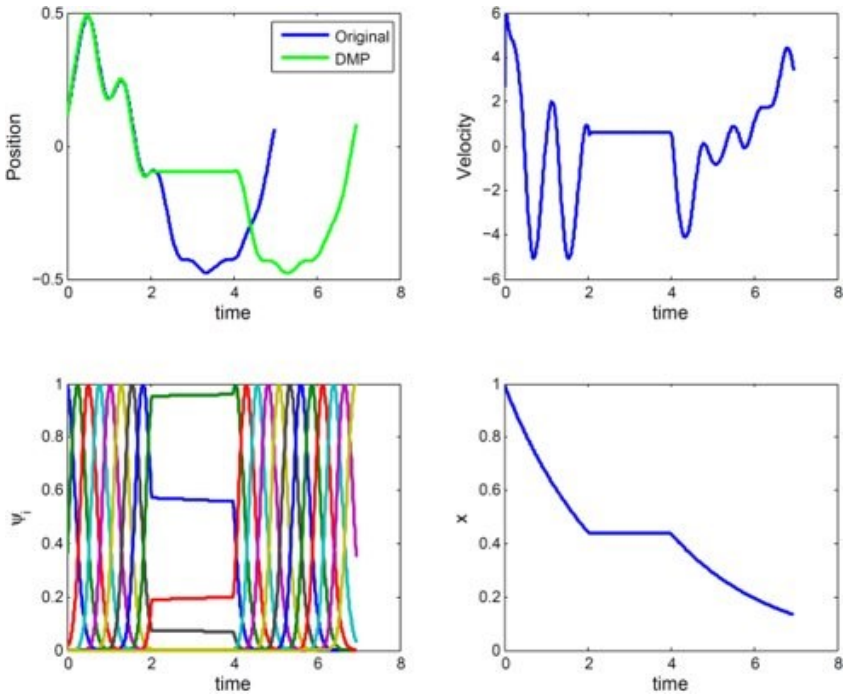
Issues

- In the classical formulation $f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x(g - y_0)$ where $(g - y_0)$ is used to ensure scale invariance and avoid high acceleration
- What happens **if $\boldsymbol{g} = \boldsymbol{y_0}$**, e.g., height coordinate in pick-and-place?
  - $f(x) = 0, z_0 = 0 \rightarrow \tau\, \dot{z}_0 = \alpha_z(\beta_z(g - y) - 0) + 0 = 0 \rightarrow$ motion does not start!
- What happens if $g \approx y_0$?
  - The term $(g - y_0) \approx 0 \rightarrow w_i \rightarrow \infty$ to reproduce the desired force $\rightarrow$ Large overshoot when you change the goal!
- Solution: $f(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x \cancel{(g - y_0)} \rightarrow f'(x) = \frac{\sum_{i=1}^{N} w_i \psi_i(x)}{\sum_{i=1}^{N} \psi_i(x)} x$

$$\tau\, \dot{z}_0 = \alpha_z(\beta_z(g - y) - z) + (g - y_0)x + f'(x)$$

  - The term $(g - y_0)x$ avoids high accelerations, preserves scale invariance, and vanishes for $t \rightarrow \infty$ as $x \rightarrow 0$ (**stability**)
  - The new forcing term $f'(x)$ does not vanish if $g = y_0$

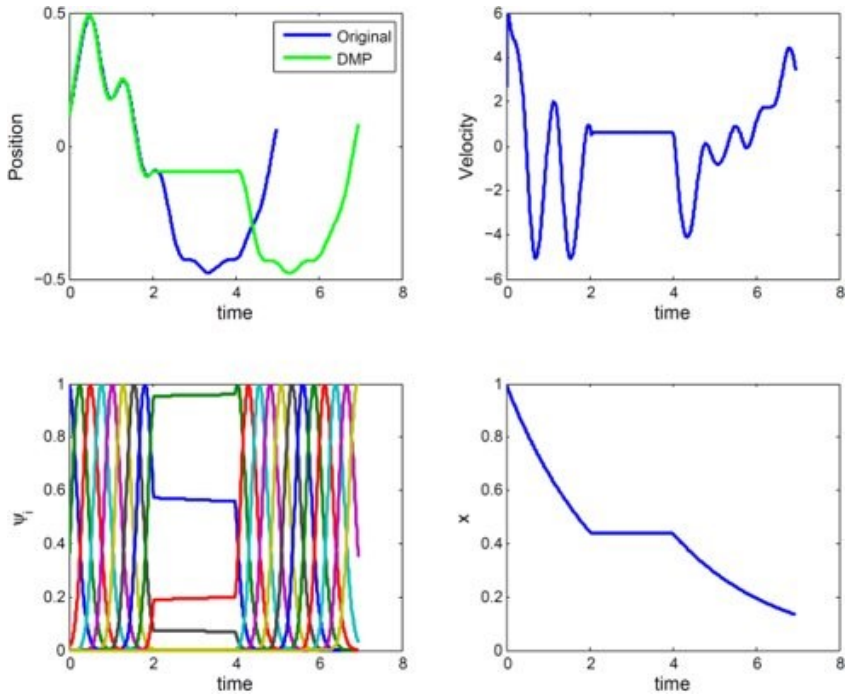
Discrete DMP
Phase stopping



- For t sufficiently large, $x \approx 0$ and the motions generated by a linear dynamics
- What if $x \approx 0$ is too early?
  - Large deformations in the generated motion
  - Can be handled by setting τ

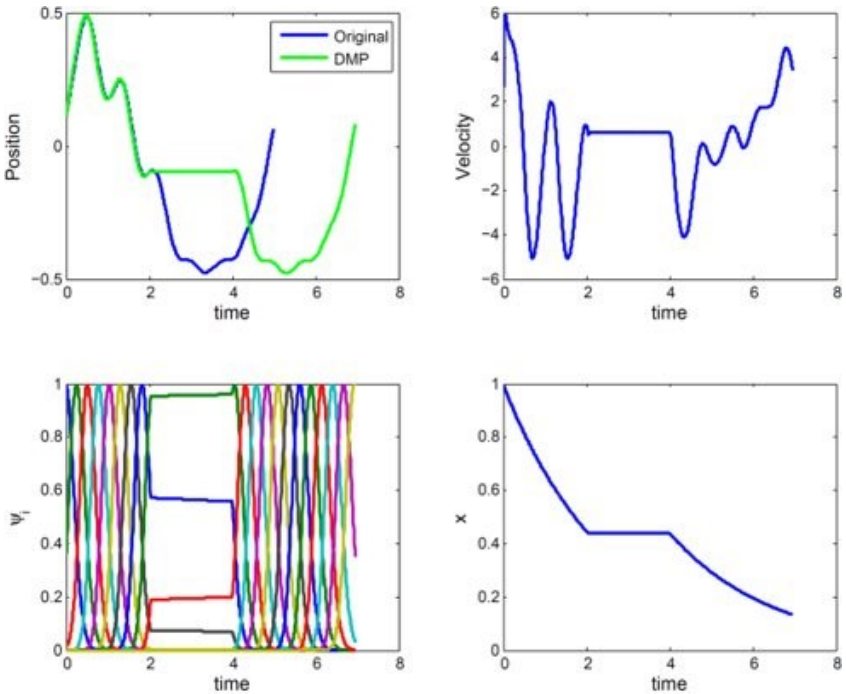
Ijspeert, A. J., Nakanishi, J., Hoffmann, H., Pastor, P., & Schaal, S. (2013). Dynamical movement primitives: Learning attractor models for motor behaviors. *Neural Computations*, *25*(2), 328–373.

Discrete DMP
Phase stopping



- For t sufficiently large, $x \approx 0$ and the motions generated by a linear dynamics
- What if $x \approx 0$ is too early?
  - Large deformations in the generated motion
  - Can be handled by setting τ
- What if the robot internal controller cannot track the motion $\alpha_{px}|\bar{y} - y|$?
  - Slow down the phase evolution to let the controller to "fill the gap"

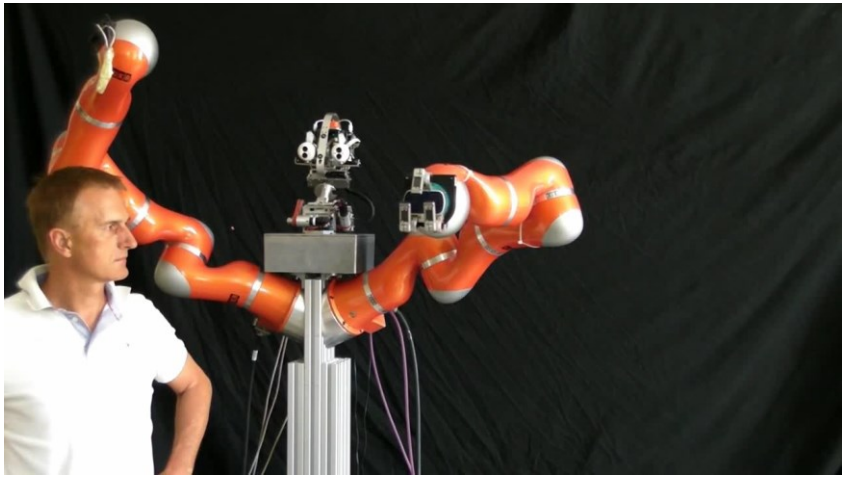
Discrete DMP
Phase stopping



- For t sufficiently large, $x \approx 0$ and the motions generated by a linear dynamics
- What if $x \approx 0$ is too early?
  - Large deformations in the generated motion
  - Can be handled by setting τ
- What if the robot internal controller cannot track the motion $\alpha_{px}|\bar{y} - y|$?
  - Slow down the phase evolution to let the controller to "fill the gap"
  - Replace the canonical system with

$$\tau\dot{x} = -\frac{\alpha_x x}{1 + \alpha_{px}\|\bar{y} - y\|}$$

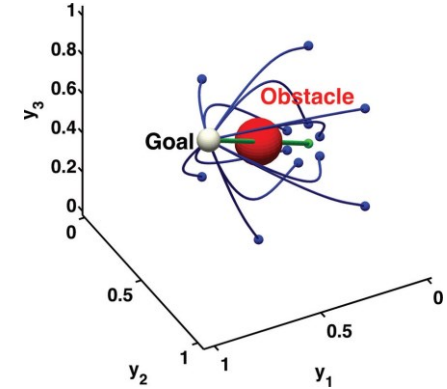
Discrete DMP
Phase stopping



- For t sufficiently large, $x \approx 0$ and the motions generated by a linear dynamics
- What if $x \approx 0$ is too early?
  - Large deformations in the generated motion
  - Can be handled by setting τ
- What if the robot internal controller cannot track the motion $\alpha_{px}|\bar{y} - y|$?
  - Slow down the phase evolution to let the controller to "fill the gap"
  - Replace the canonical system with
$$\tau\dot{x} = -\frac{\alpha_x x}{1 + \alpha_{px}\|\bar{y} - y\|}$$
  - Experimental evidences show that using $\tau\dot{y} = 1 + \alpha_{py}(\bar{y} - y)$ speeds up the error reduction

DMPs

Obstacle avoidance design: Spatial coupling

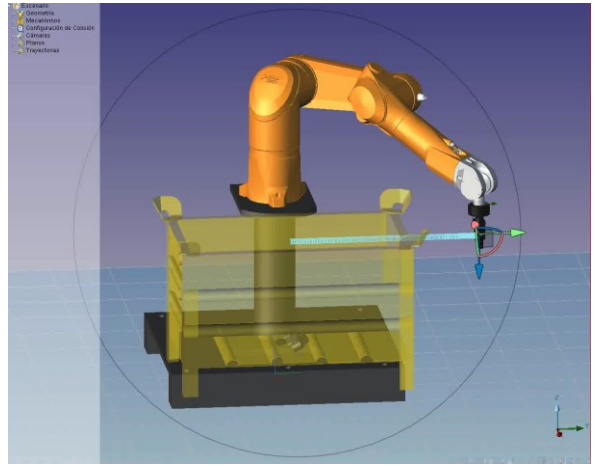


- Given a Cartesian position $\mathbf{y} = [y_1, y_2, y_3]^{\mathrm{T}}$ and velocity $\mathbf{z} = [z_1, z_2, z_3]^{\mathrm{T}}$, an obstacle placed at $\mathbf{o} = [o_1, o_2, o_3]^{\mathrm{T}}$. Then we add a coupling term $\mathbf{C}_t = [c_1, c_2, c_3]$ to DMP

$$\tau \dot{z} = \alpha_z(\beta_z(g - y) - z) + f(x) + \mathbf{C}_t,$$

where $\mathbf{C}_t = \gamma \mathbf{R} \dot{\mathbf{y}} \theta \exp(-\beta\theta)$

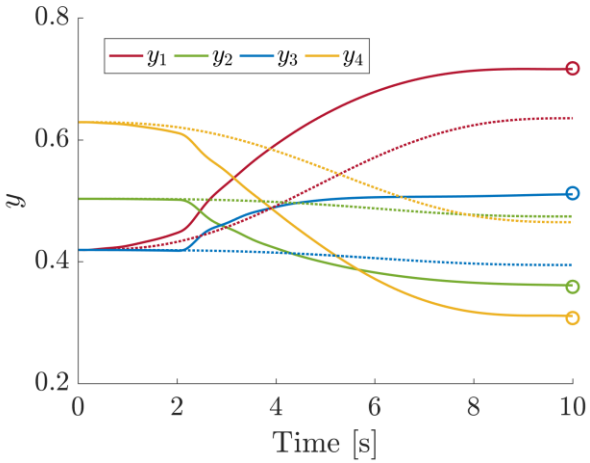
- $\theta = \arccos\left(\frac{(\mathbf{o}-\mathbf{y})^{\mathrm{T}}\dot{\mathbf{y}}}{\|\mathbf{o}-\mathbf{y}\|\|\dot{\mathbf{y}}\|}\right)$,
- θ is the angle between $\dot{\mathbf{y}}$ and the direction approaching the obstacle $(\mathbf{o} - \mathbf{y})$
- $\mathbf{R}$ is a rotation matrix computed from the axis-angle representation with $\mathbf{r} = (\mathbf{o} - \mathbf{y}) \times \dot{\mathbf{y}}$.
- $\theta \exp(-\beta\theta)$ is a variable steering direction: large for $\theta \approx 0°$, small for $\theta \to \pm 90°$

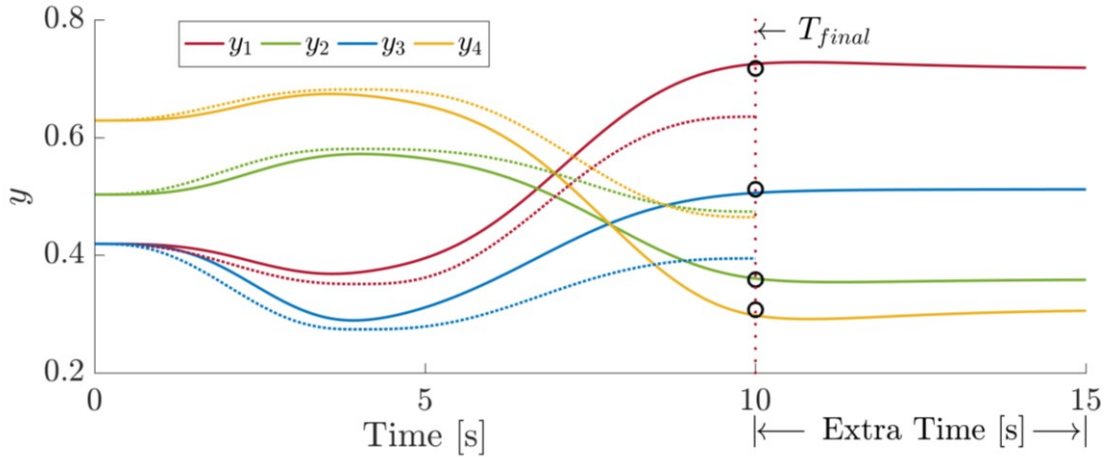


- **Multiple obstacles** $\mathbf{o}_1, \mathbf{o}_2, \ldots \rightarrow \mathbf{C}_t = \mathbf{C}_{t,1} + \mathbf{C}_{t,2} + \cdots$

DMPs
Goal switching

- Adaptation to new goal attractor g $\tau \dot{g} = \alpha_g (g_{new} - g)$





DMPs

Implementations

- DMP code collection https://gitlab.com/dmp-codes-collection
  - Contains several implementations of DMP in Matlab, Python, or C++
    - Discrete and periodic
    - Riemannian manifolds: Unit quaternion and Symmetric Positive Definite matrix
    - Interaction and coupling terms

References and good Readings

- Saveriano, Matteo, Fares J. Abu-Dakka, Aljaz Kramberger, and Luka Peternel. "***Dynamic Movement Primitives in Robotics: A Tutorial Survey***" arXiv preprint arXiv:2102.03861 (2021).
- A.G. Billard, S. Calinon, R. Dillmann, Learning from Humans. In: B. Siciliano, O. Khatib (eds) Springer Handbook of Robotics. Springer Handbooks. Springer, Cham., 2016 (DOI: 10.1007/978-3-319-32552-1_74)
- A. J. Ijspeert, J. Nakanishi, H. Hoffmann, P. Pastor, S. Schaal, Dynamical Movement Primitives: Learning Attractor Models for Motor Behaviors, Neural Computation, 25(2):328—373, 2013
- S. Calinon, Robot programming by demonstration: a probabilistic approach, CRC Press, 2009 (ISBN 978-1-4398-0867-2)

Summary

- Robot learning is essential in order to make robots to execute new tasks and avoid hard-coding.
- Learning from demonstration provides a friendly way to teach robots from human.
- Dynamic movement primitive is one of the imitation learning techniques that can be used to teach robots manipulation skills from single demonstration.